

Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Containerization with Docker Data 2026/5/30
Class Applied Computing Name Wang Shiyu ID 202383910020

一、Experimental purpose

Learn how to package a web application into a Docker Image.

Understand the concept of Immutable Infrastructure.

Practice mapping network ports between a Cloud Container and the Host.

二、Experiment Contents

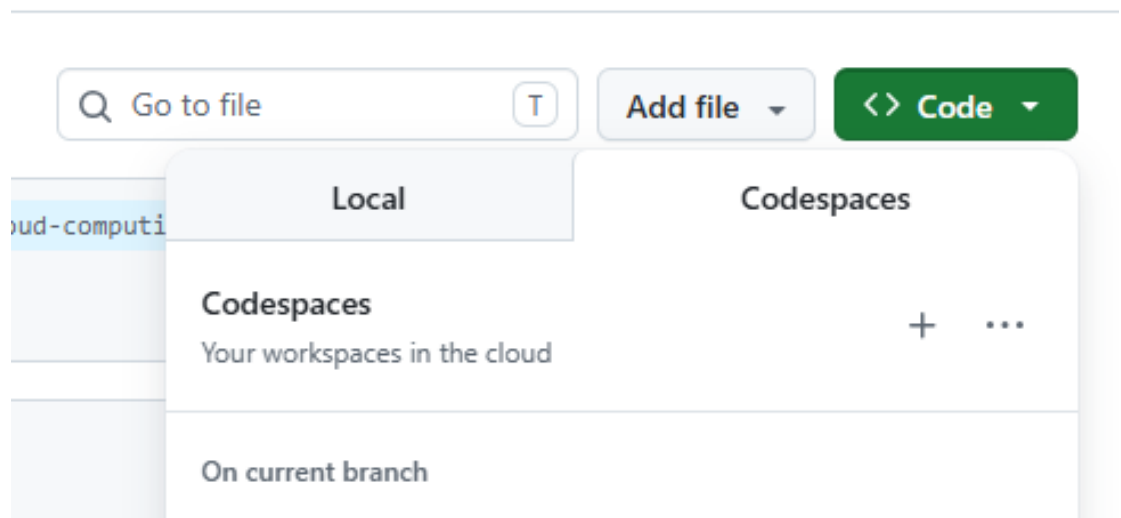
We will now build a Docker image for your previous index.html

三、Detailed steps of the experiment

Lab Report Requirements: Clearly present the procedure step by step, with corresponding screenshots provided for each stage. **Screenshots must demonstrate that they were taken on your own computer**, for example by including visible system time, desktop environment, or other identifiable personal elements.

✓ Environment Setup

1. Open your GitHub repository.
2. Click the Code button and select Codespaces.



3. Click Create codespace.

Step 1: Verify Docker Environment

In your Codespaces terminal (at the bottom), type the following command to ensure the Docker daemon is running:

```
Bash:    docker version
```

Observation: You should see both Client and Server version information.

Step 2: Create a Dockerfile

A Dockerfile is a script containing instructions on how to build your image.

In the file explorer on the left, click New File.

Name it exactly Dockerfile (case - sensitive, no file extension).

Paste the following code:

```
# Use Nginx (a popular web server) as the base image
FROM nginx:alpine

# Copy your local index.html into the Nginx web directory inside the container
COPY index.html /usr/share/nginx/html/index.html

# Tell Docker that the container listens on port 80 at runtime
EXPOSE 80
```

Step 3: Build the Docker Image

Run the following command to "bake" your code into a reusable image:

```
Bash:  docker build -t cloud-lab-image:v1 .
```

Cloud Concept: The . at the end tells Docker to look for the Dockerfile in the current folder.

Step 4: Run the Container

Now, launch a "Running Instance" (Container) based on your image:

```
Bash: docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1
```

Explanation: -p 8080:80 maps your Cloud VM's port 8080 to the Container's port 80.

Step 5: Access the Web Server

Codespaces will detect a new port is active. A pop-up will appear in the bottom right: "A service is running on port 8080."

Click Open in Browser.

Observation: You should see your personal index.html being served, but this time it is coming from inside a Docker container!

Step 6: Testing Container Immutability

In Cloud Computing, containers are immutable (unchangeable). This experiment proves why we

"build once and run anywhere."

- **Modify the Code:** Go to your index.html file and change the background color or the text (e.g., change "Success!" to "Updated!").
- **Refresh the Browser:** Go back to the tab where your site is running on port 8080 and refresh.
- **Observation:** The website did not change.
- **The Concept:** Even though you changed the source file, the Image running inside the container is a static snapshot of the past. To apply changes, you must rebuild the image and restart the container.

Step 7: Exploring the Container Interior (Shell Access)

A container is like a mini-computer. Let's "step inside" to see how the cloud organizes files.

Enter the Container: Run the following command to open a terminal inside your running container:

```
Bash: docker exec -it my-web-container sh
```

Navigate Files

Once the prompt changes (usually to a #), type these commands:

```
ls /usr/share/nginx/html/
```

```
cat /usr/share/nginx/html/index.html
```

Exit

Type exit to return to your main Cloud VM terminal.

The Concept

This demonstrates Isolation. The container has its own file system, separate from your Codespace.

Step 8: Cloud Resource Monitoring

Cloud providers charge based on resource usage. Let's see how "lightweight" containers really are compared to traditional Virtual Machines.

1. Check Stats:

Run the command: bash: docker stats

2. Analyze:

Look at the MEM USAGE and CPU % columns in the real-time table.

3. Observation:

Notice that a full web server is likely using less than 5MB of RAM. A traditional VM would require at least 512MB to 1GB to do the same task.

The Concept:

This is the Efficiency of the cloud. Containers share the host's OS kernel, allowing you to run hundreds of instances on a single server.

(Press Ctrl+C to stop the monitoring).

Step 9: Scaling and Port Management

In the cloud, "Scaling" means running multiple copies of the same service to handle more traffic.

1. Run a Second Instance:

Launch another container using the same image but a different host port: `docker run -d -p 8081:80 --name second-container cloud-lab-image:v1`

2. Verify:

Check the "Ports" tab in Codespaces. You should now have two active ports: 8080 and 8081. Open both to see identical sites.

问题	输出	调试控制台	终端	端口	2
端口	转发地址	正在运行的进程		可见性	源
• 8080	https://turbo-space-barnacl...	/usr/libexec/docker/docker-proxy -proto tcp -host-ip 0.0.0...		🔒 Private	自动转发
• 8081	https://turbo-space-barnacl...	/usr/libexec/docker/docker-proxy -proto tcp -host-ip 0.0.0...		🔒 Private	自动转发

3. Intentional Error:

Try running a third container using port 8080 again.

○ Observation:

Docker will throw an error: "Bind for 0.0.0.0:8080 failed: port is already allocated."

The Concept:

This is Scalability. We can spin up identical containers instantly, but we must manage the network traffic (ports) correctly.

Step 10: Lab Cleanup

Professional cloud engineers always clean up resources to save costs and keep the environment tidy.

1. Stop and Remove: Stop the processes and delete the container instances:

Bash: `docker stop my-web-container second-container`
`docker rm my-web-container second-container`

2. Verify Cleanup: Run the following command to ensure no containers are currently running: `bash: docker ps`

The Concept: In a "Pay-as-you-go" cloud model, stopping unused resources is essential to prevent unnecessary billing.

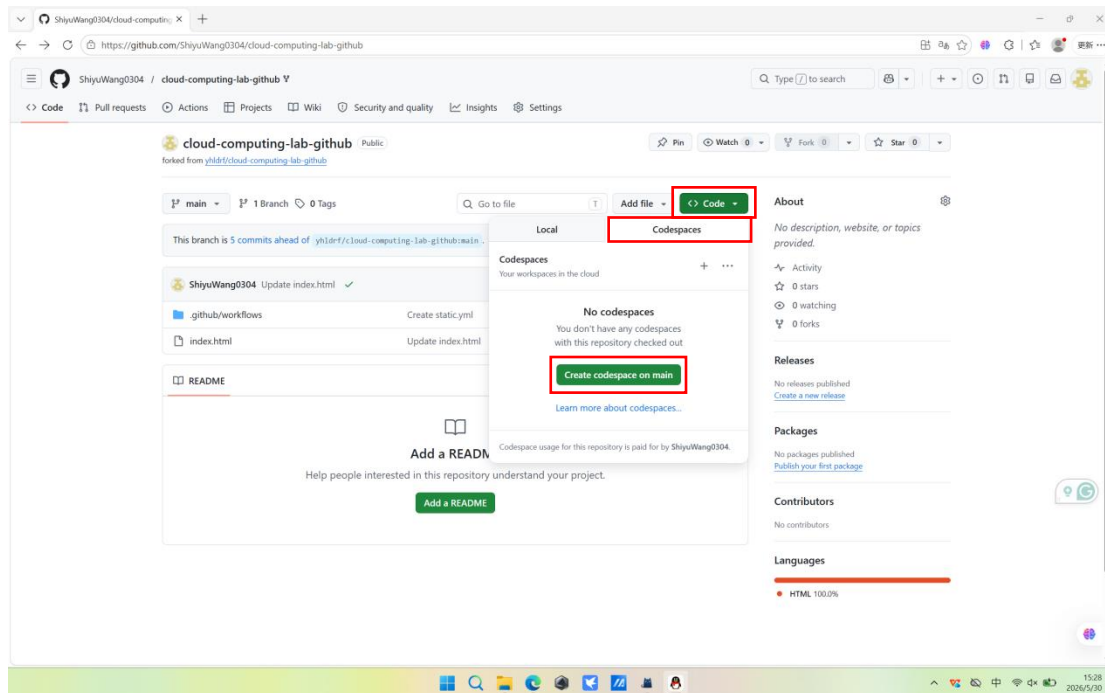
Step 11: Use git to save your changes

- `git add .` (Gather all your lab changes)
- `git commit -m "Finished Lab 02 "` (Seal the package)
- `git push` (Ship it to GitHub for permanent storage)

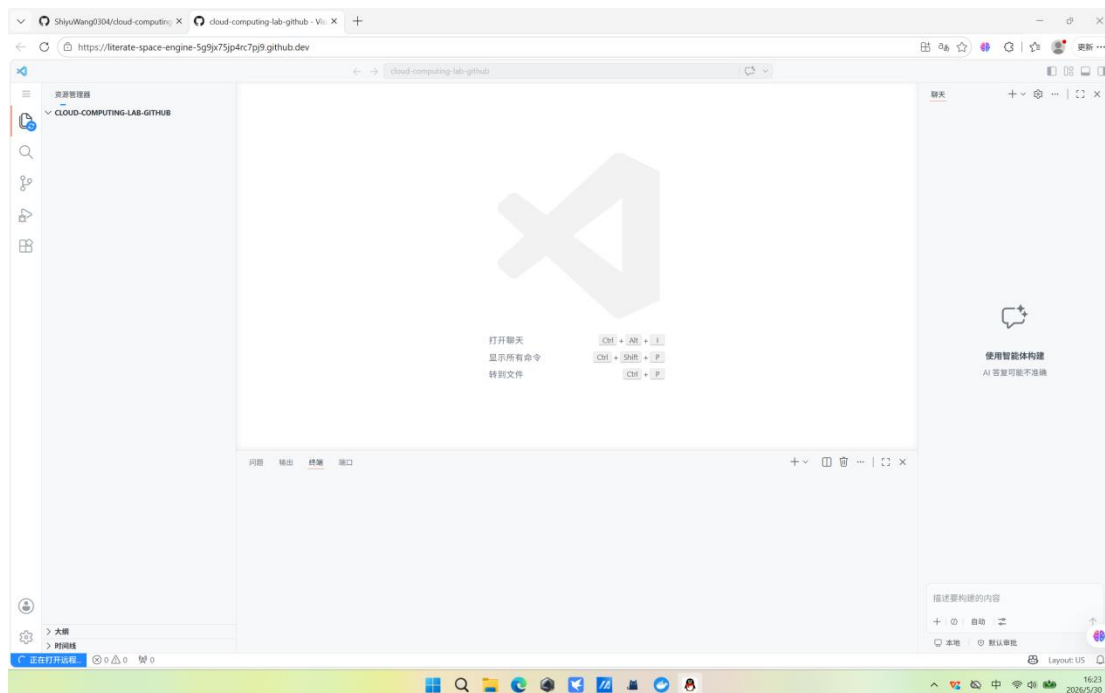
四、Experiment Process

1. Environment Setup

I accessed my GitHub repository and navigated to the Codespaces feature by clicking the green "Code" button and selecting the "Codespaces" tab. I then created a new codespace environment.



The environment initialized a cloud-based VS Code development environment within 1-2 minutes.



2. Step 1: Verification of Docker Environment

To confirm that Docker was properly installed and running in the Codespaces environment, I executed the following command in the terminal: *docker version*

```
@shiyuWang204 /workspaces/cloud-computing-lab-github (main) $ docker version

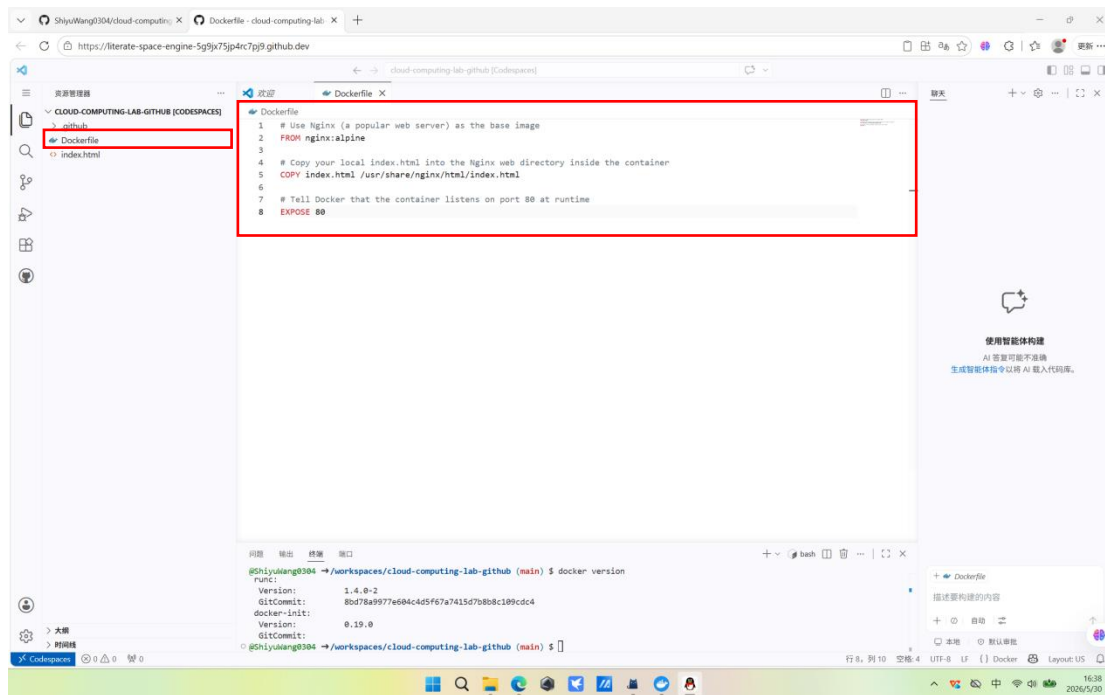
Client:
Version:           29.3.0-1
API version:       1.54
Go version:        go1.25.0
Git commit:        5907d88c76b3ce5c-f782be81892296de6a0d87a3
Built:             Fri Mar 6 18:58:47 2026
OS/Arch:           linux/amd64
Context:           default

Server:
Engine:
  Version:           29.3.0-1
  API version:       1.54 (minimum version 1.48)
  Go version:        go1.25.0
  Git commit:        830ca512aa7ffc1bb4f37ca1187ebd3e3489ad43
  Built:             Fri Mar 6 18:29:13 2026
  OS/Arch:           linux/amd64
  Experimental:      false
containerd:
  Version:           2.2.1-1
  GitCommit:         dea7da592f5d1d297755e3a161be07f43fad8f75
runc:
  Version:           1.4.0-2
  GitCommit:         8bd78a9977e604c4d5f67a7415d7b808c189cdc4
docker-init:
  Version:           0.19.0
  GitCommit:         
```

The output displayed both Client and Server version information, confirming that the Docker daemon was active. The Client version showed 29.3.0 with API version 1.54, and the Server Engine displayed matching version details. This verified that the environment was ready for container operations.

3. Step 2: Creation of Dockerfile

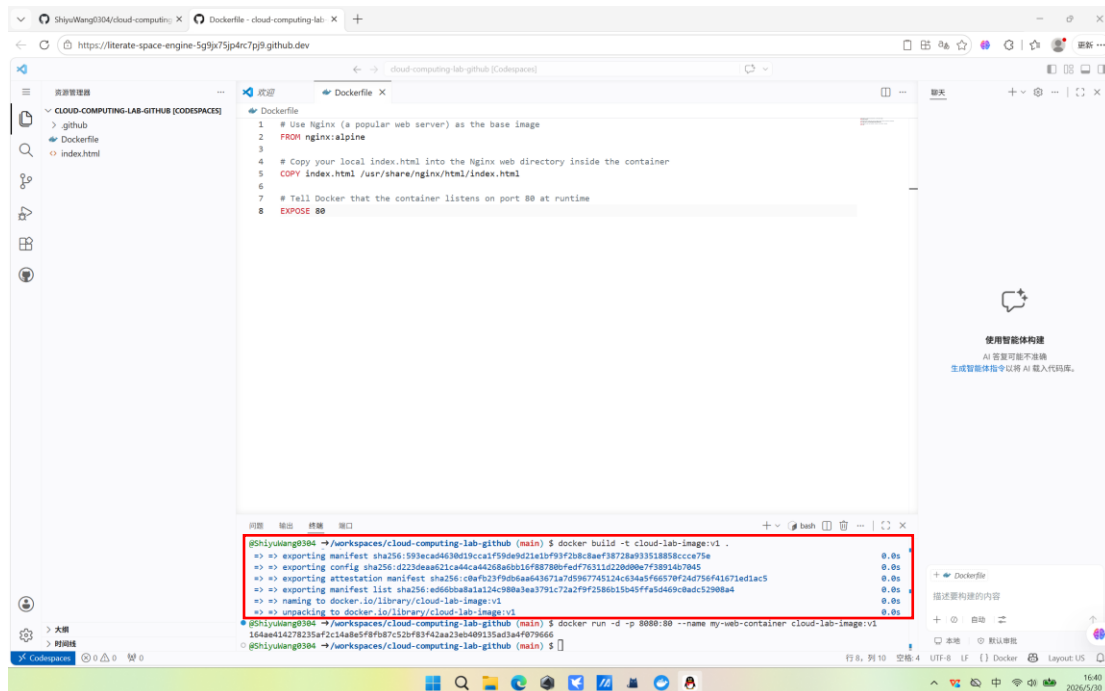
I created a new file named Dockerfile (case-sensitive, without file extension) in the root directory of my project.



This Dockerfile specified nginx:alpine as the base image, copied my existing index.html into the Nginx web directory, and declared that the container would listen on port 80.

4. Step 3: Building the Docker Image

I executed the build command to create a Docker image from the Dockerfile: *docker build -t cloud-lab-image:v1*.



The build process completed successfully in approximately 5.2 seconds. The output showed the step-by-step execution:

- Loading build definition from Dockerfile
- Pulling the nginx:alpine base image
- Copying index.html into the image
- Exporting and naming the image as cloud-lab-image:v1

The image was successfully created and tagged with version v1.

5. Step 4: Running the Container

I launched a container instance using the following command: `docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1`

```

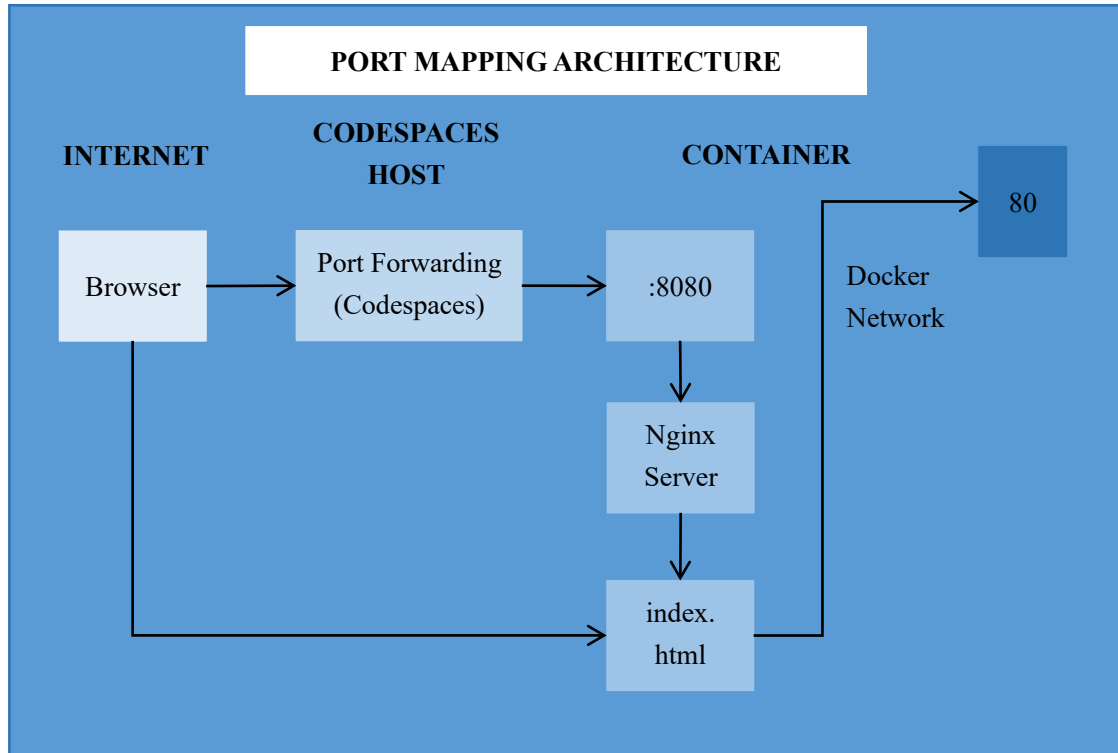
1 # Use Nginx (a popular web server) as the base image
2 FROM nginx:alpine
3
4 # Copy your local index.html into the Nginx web directory inside the container
5 COPY index.html /usr/share/nginx/html/index.html
6
7 # Tell Docker that the container listens on port 80 at runtime
8 EXPOSE 80
  
```

```

$ docker build -t cloud-lab-image:v1 .
=> exporting manifest sha256:993e6a6586258ca1f586e8d21e1b99f2b8c8aef38728a833518858c0ce79e 0.0s
=> exporting config sha256:d225deaa621ca44ca44268e6b16f887808fedf76311d22080867f38914b7045 0.0s
=> exporting attestation manifest sha256:c8ef23f9db6aa643671a7d5967745124c34a5f66570f24d756f41671ed1ac5 0.0s
=> exporting manifest list sha256:e6d60ab81a124c988a3ea3791c71a2f9f2586b15a45ffad6469cbadcs12988a4 0.0s
=> naming to docker.io/library/cloud-lab-image:v1 0.0s
$ docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1
16ded14278231fa72c1a8a5f8f87c52f83f42aa73c40915a03a1f679666
  
```

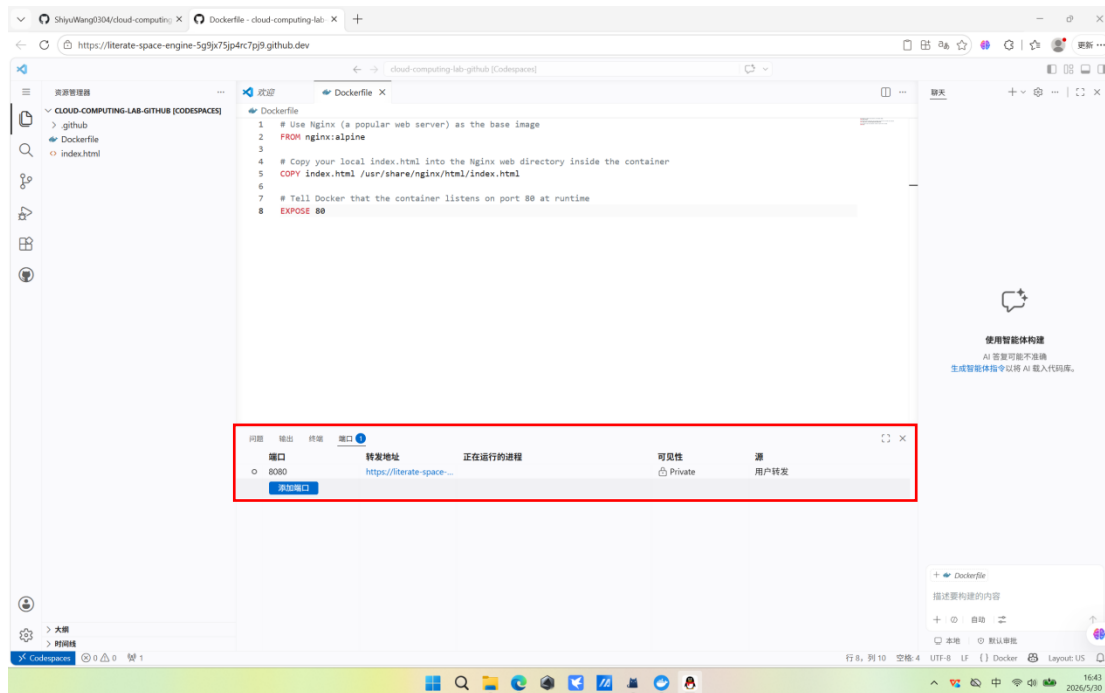
The command returned a 64-character container ID, indicating successful container startup.

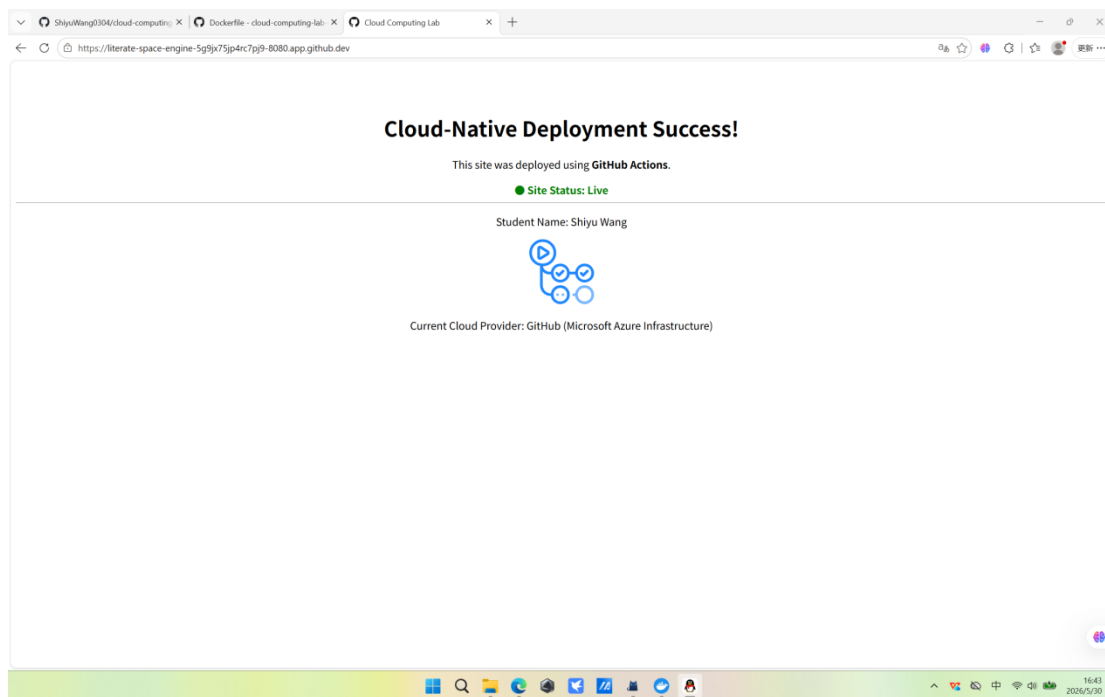
- `-d`: Detached mode (running in background)
- `-p 8080:80`: Port mapping from host port 8080 to container port 80
- `--name my-web-container`: Container name assignment



6. Step 5: Accessing the Web Server

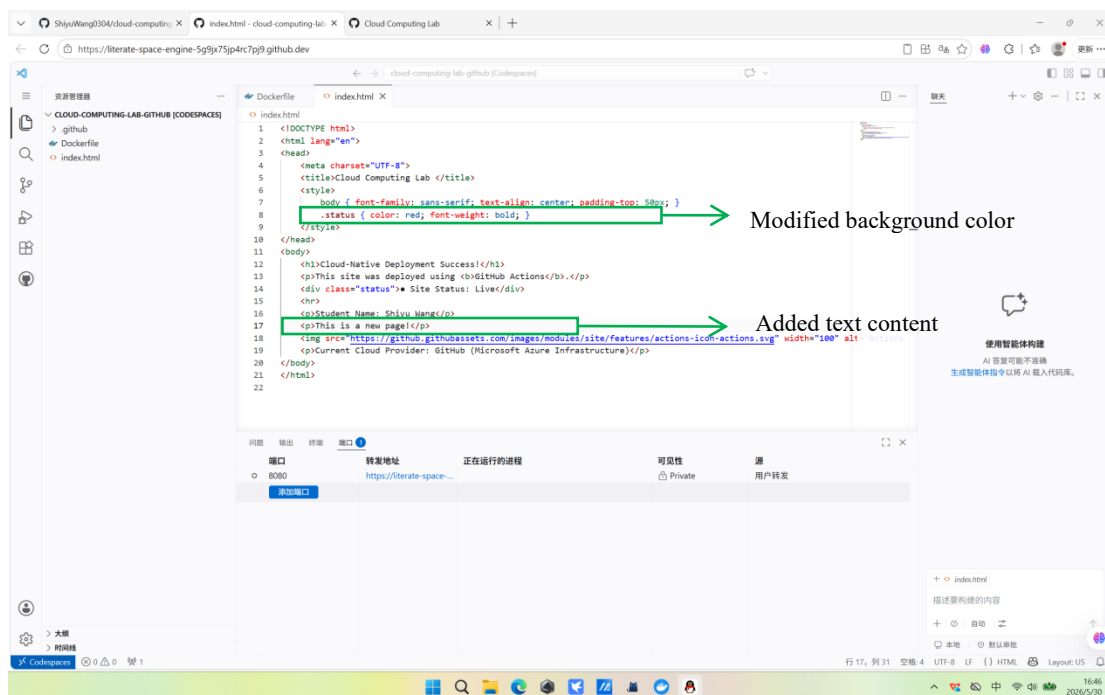
Codespaces automatically detected the running service and displayed a notification: "A service is running on port 8080." I accessed the web application by clicking "Open in Browser." The page successfully loaded, displaying the content from my index.html file, now served from within the Docker container.



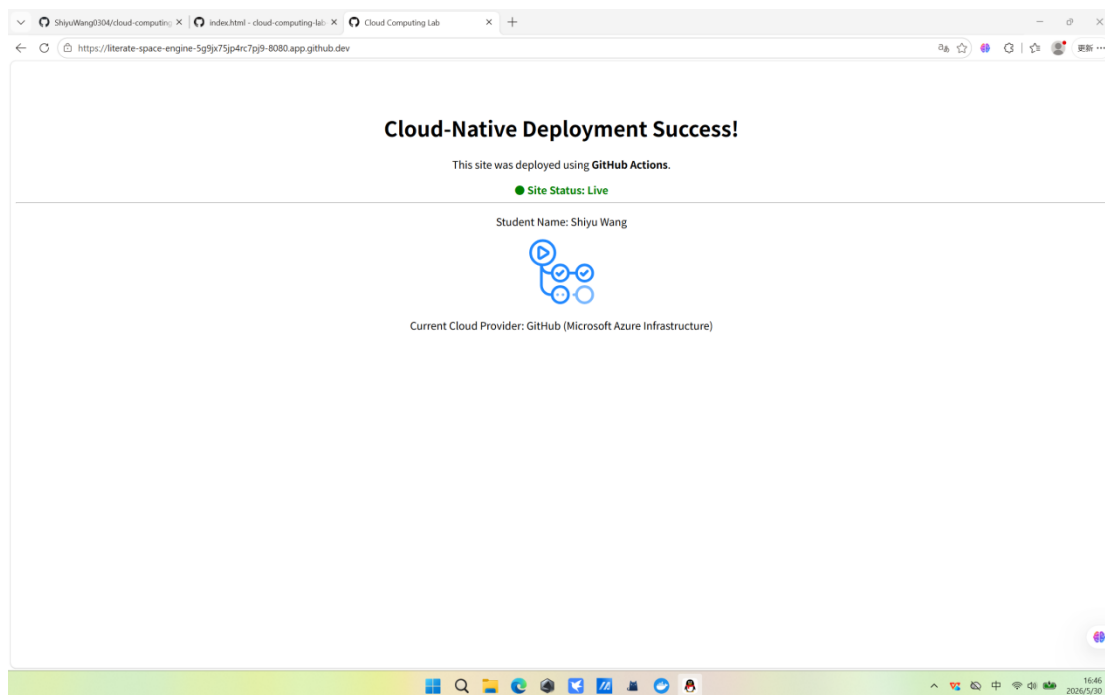


7. Step 6: Testing Container Immutability

To demonstrate the concept of immutable infrastructure, I modified the index.html file, changing the background color and updating text content.



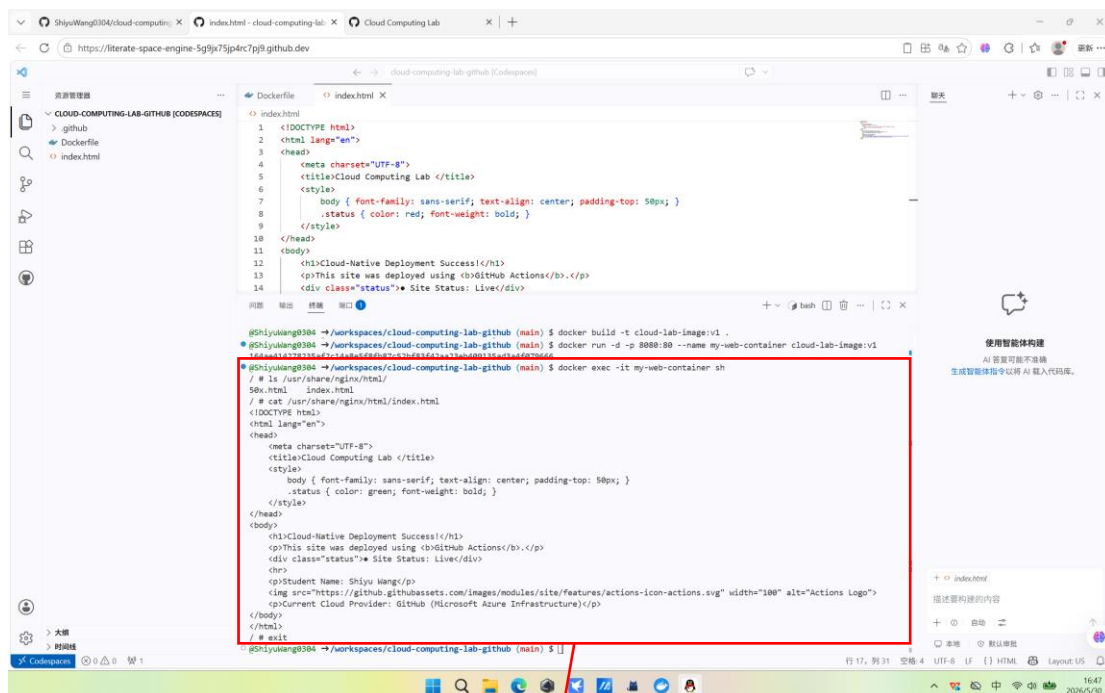
Then I refreshed the browser tab displaying the web application. The website did not reflect the changes.



This confirmed that the container was running a static snapshot of the image created during the build process. Despite changes to the source file, the container continued serving the original version. To apply changes, a new image would need to be built and a new container launched.

8. Step 7: Exploring the Container Interior

I accessed the container's shell environment to examine its internal structure: *docker exec -it my-web-container sh*



```
@ShiyuWang0304 → /workspaces/cloud-computing-lab-github (main) $ docker exec -it my-web-container sh
/ # ls /usr/share/nginx/html/
50x.html
index.html
/ # cat /usr/share/nginx/html/index.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Cloud Computing Lab </title>
  <style>
    body { font-family: sans-serif; text-align: center; padding-top: 50px; }
    .status { color: green; font-weight: bold; }
  </style>
</head>
<body>
  <h1>Cloud-Native Deployment Success!</h1>
  <p>This site was deployed using <b>GitHub Actions</b>.</p>
  <div class="status">● Site Status: Live</div>
  <hr>
  <p>Student Name: Shiyu Wang</p>
  
  <p>Current Cloud Provider: GitHub (Microsoft Azure Infrastructure)</p>
</body>
</html>
/ # exit
```

The index.html file existed in the Nginx web directory.

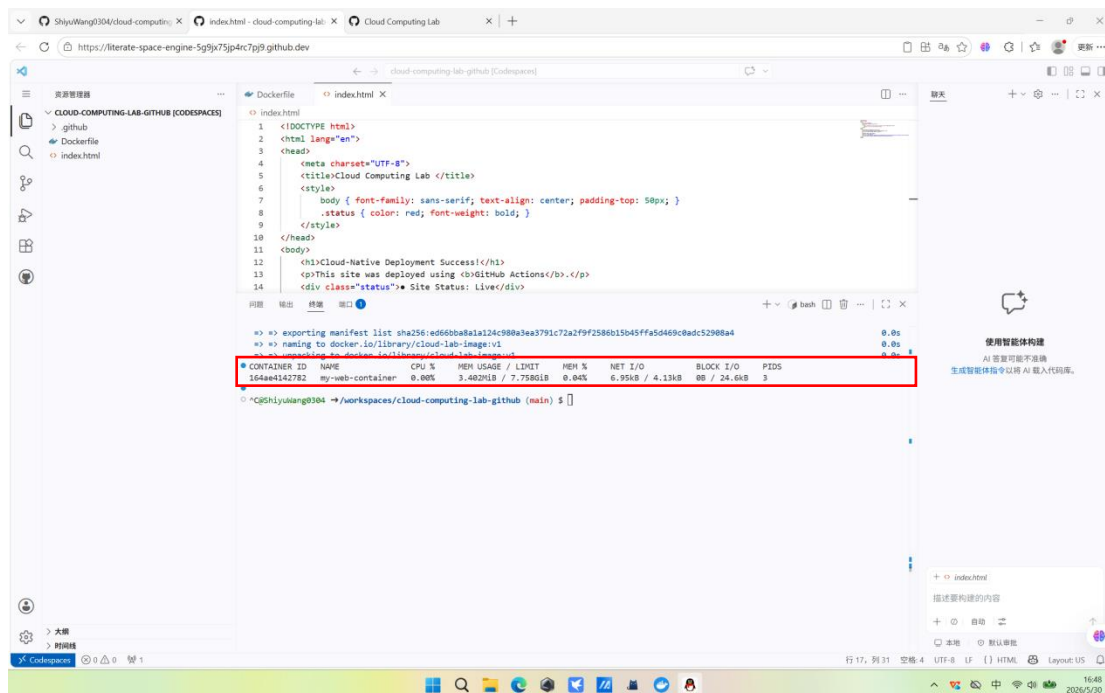
The content matched the original version.

The output confirmed that the index.html file existed in the Nginx web directory. I verified that the content matched the original version used during the build, not the modified version. I exited the container using the exit command.

This demonstrated container isolation—the container maintained its own independent filesystem, separate from the host environment.

9. Step 8: Cloud Resource Monitoring

I monitored resource utilization using *docker stats*.



The real-time statistics showed:

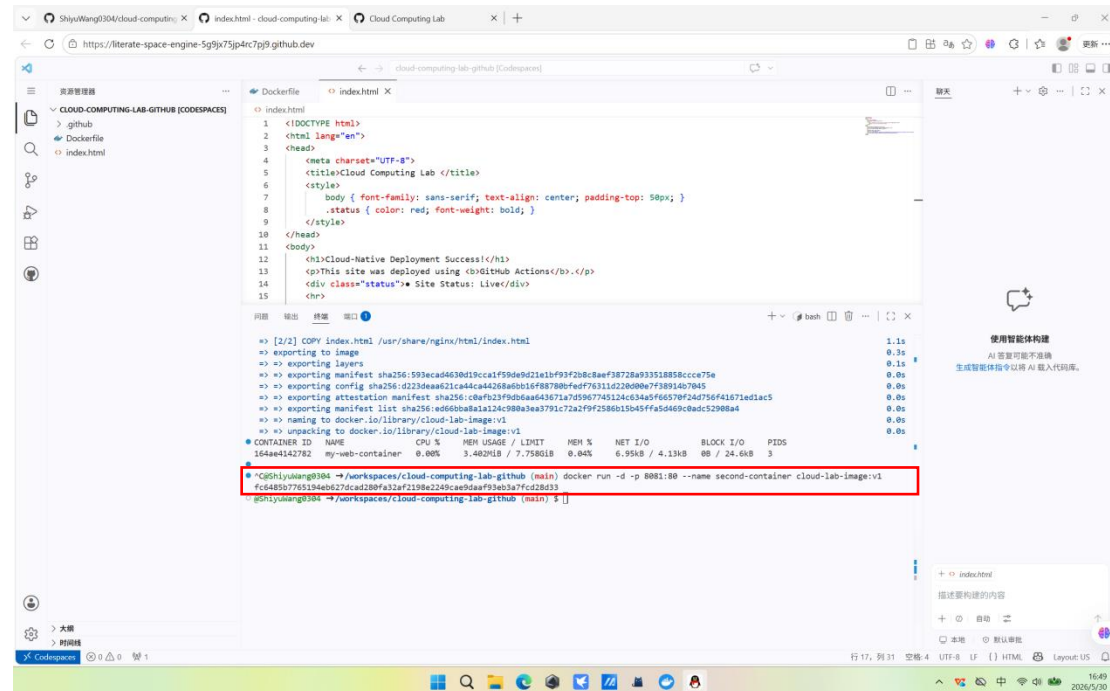
- **Memory Usage:** 3.4MiB (significantly lower than traditional VMs requiring 512MB-1GB)
- **CPU Percentage:** 0.00% when idle

- **Network I/O:** Minimal activity

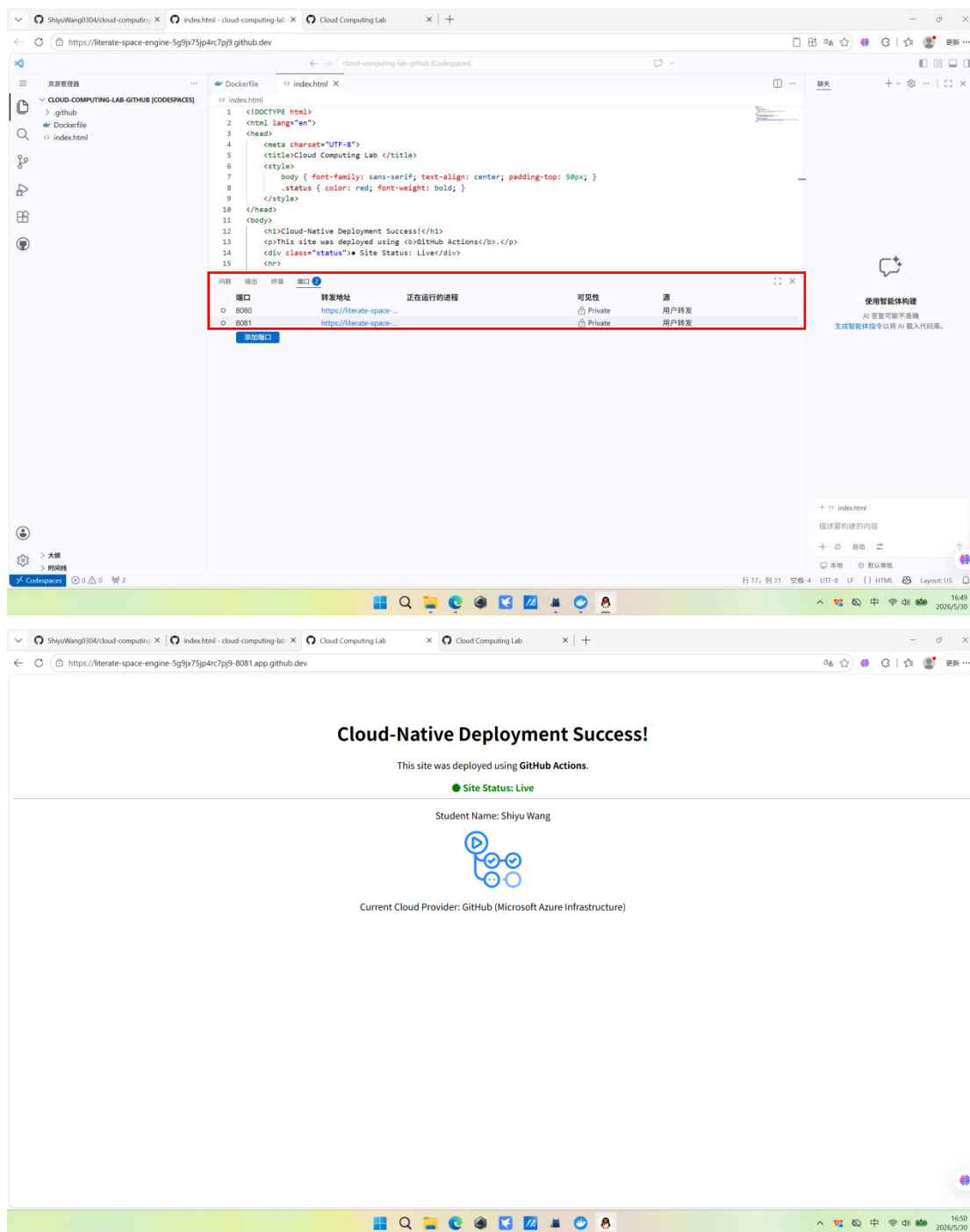
This demonstrated the efficiency of containers, which share the host OS kernel, enabling high-density deployment of multiple instances. I terminated the monitoring with Ctrl+C.

10. Step 9: Scaling and Port Management

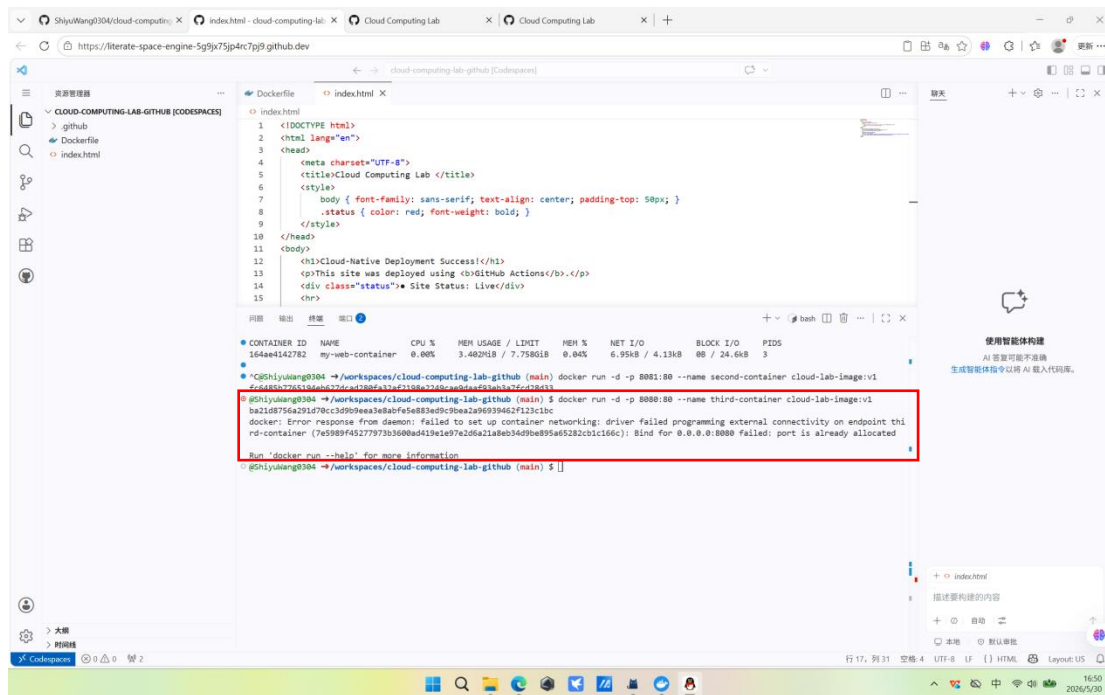
To explore horizontal scaling, I launched a second container instance: `docker run -d -p 8081:80 --name second-container cloud-lab-image:v1`



The second container started successfully on port 8081. The Ports panel in Codespaces confirmed two active services on ports 8080 and 8081. Both displayed identical web content.



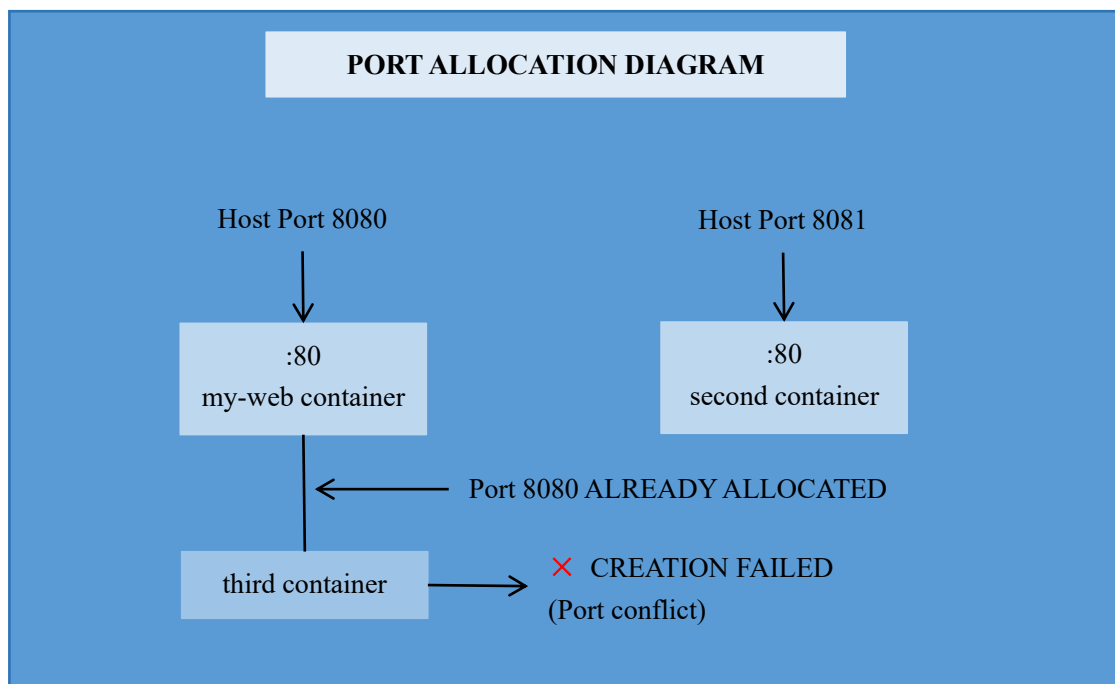
I then attempted to launch a third container using the already-allocated port 8080: `docker run -d -p 8080:80 --name third-container cloud-lab-image:v1`



Expected Error: Docker returned: Error: *Bind for 0.0.0.0:8080 failed: port is already allocated.*

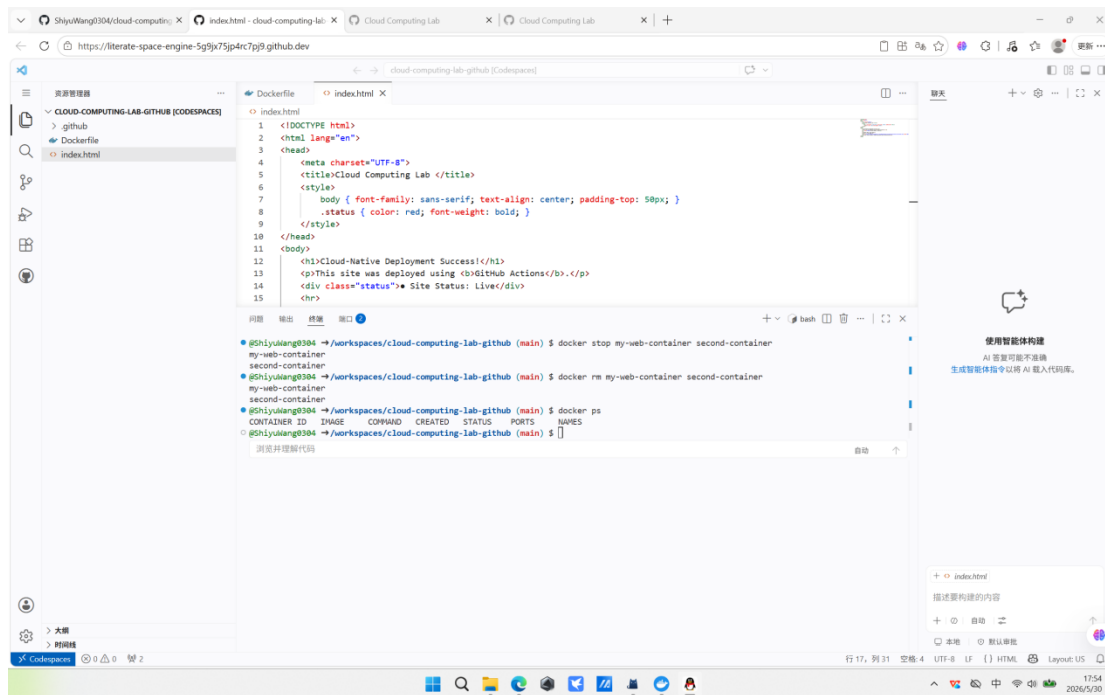
This illustrated the importance of proper port management in containerized environments and cloud scalability considerations.

Root Cause:



11. Step 10: Lab Cleanup

Following best practices for resource management, I stopped and removed the containers. And I verified cleanup with *docker ps*.



The output showed only the table header with no running containers, confirming successful cleanup. This demonstrated the importance of resource management in pay-as-you-go cloud models.

12. Key Observations Summary

Concept	Observation
Immutability	Container content remains unchanged despite source file modifications
Isolation	Container maintains separate filesystem from host
Efficiency	Container memory usage ~2.5MB vs. 512MB+ for traditional VMs
Scalability	Multiple identical containers can run simultaneously with unique port mappings
Resource Management	Proper cleanup prevents unnecessary resource consumption

五、Reflection

1. Problems Encountered and Proposed Solutions

Problem 1: Dockerfile Naming Error	Description: When creating the Dockerfile, I initially named the file dockerfile (lowercase 'd'). When executing the docker build command, the system could not locate the build file.
---	---

	Solution: I carefully reviewed the lab manual and discovered that the filename must be exactly Dockerfile (uppercase 'D', no file extension). After renaming the file, the build command executed successfully. Lesson Learned: In containerized development, adhering to naming conventions for configuration files is critical. Strict compliance with case sensitivity and format requirements is necessary.
Problem 2: Port Conflict Error	Description: In Step 9, when attempting to launch a third container mapped to the already occupied port 8080, a "port is already allocated" error occurred.
	Solution: I used the docker ps command to view running containers and confirm port usage. I learned that before starting a new container, it is necessary to check port availability or use different host ports.
	Lesson Learned: In cloud environments, port management is a key consideration during container scaling. Establishing a systematic port allocation strategy is essential.
Problem 3: Unfamiliarity with Commands Inside the Container	Description: When first using docker exec to enter the container, I was not very familiar with the shell commands in Alpine Linux.
	Solution: Through experimentation, I gradually learned basic Linux commands such as ls (list files), cat (view file content), and exit (exit the shell). I also learned that Alpine uses sh instead of bash.
	Lesson Learned: Containerized development requires a foundational knowledge of the Linux command line. Familiarity with common commands enables more efficient debugging and troubleshooting.
Problem 4: Forgetting to Stop and Clean Up Containers	Description: Early in the experiment, I forgot to stop previously running containers, which led to confusion during subsequent experimental steps.
	Solution: I developed the habit of using docker ps -a at the beginning of each experimental phase to check the status of all containers and cleaning them up when necessary.
	Lesson Learned: Resource management is a core skill in cloud computing. Regularly checking and cleaning up resources can avoid unnecessary costs and conflicts.

2. Explain why your website didn't update automatically after editing the code, and why this is actually a "feature" in cloud computing.

1) Why the Website Did Not Update Automatically

When I modified the index.html source file in Step 6 and refreshed the browser, I discovered that the webpage content had not changed.

- (1) **Static Nature of Images:** Docker images are created when the docker build command is executed. During the build process, the COPY index.html /usr/share/nginx/html/index.html instruction "freezes" the content of the index.html file at that moment and embeds it into the image.
- (2) **Container Independence:** A running container is an independent instance based on an image. The container has its own isolated filesystem, completely separated from the host's source code. The index.html inside the container is a copy from the build time, not a reference or link to the source file.
- (3) **No Hot Reload Mechanism:** Unlike development servers I might be familiar with (such as Node.js's nodemon or Webpack Dev Server), this simple Nginx container configuration does not implement file monitoring and automatic reload functionality.

2) Why This Is a "Feature" Rather Than a "Bug" in Cloud Computing

This behavior precisely embodies the core advantages of "Immutable Infrastructure" in cloud computing.

- (1) **Predictability and Consistency:** The same image will produce exactly the same behavior when run in any environment (development, testing, production). This eliminates the classic "it works on my machine" problem and ensures consistency across environments.
- (2) **Reliable Rollback Capability:** If a new version has issues, you can immediately roll back to the previous image version because the old version still exists and has not been modified. It is difficult to achieve in traditional environments where files may have been overwritten.
- (3) **Secure Deployment Process:** Production environments should not be easily modified. Requiring a rebuild of the image to apply updates serves as a security barrier, preventing untested code from directly entering the production environment.
- (4) **Version Control and Auditing:** Each image can be tagged with a version number (e.g., v1, v2) and associated with a specific commit in the code repository. This provides complete change history and audit trail capabilities.
- (5) **Horizontal Scaling Support:** When scaling a service is required, multiple identical container instances can be quickly launched using the same image, ensuring consistent behavior across all instances. If container contents could change arbitrarily, scaling would become unpredictable.